



Introduction

What is Vex?

Vex is a versatile, accessible, and user-friendly programming language designed for a wide range of applications, from embedded systems to the development of robust, scalable tools. It is engineered to maintain consistency and efficiency across diverse deployment environments.

Programming Model

Vex is an interpreted language that prioritizes readability and maintainability. Its syntax utilizes indentation to define structure rather than traditional brackets, and employs a clear, English-based syntax to enhance code clarity and reduce complexity.

Design Philosophy

Vex is designed to address specific gaps in modern development requirements, offering a balanced approach between versatility and specialization. Its highly scalable nature allows it to serve as either the foundational core of a project or a lightweight, modular component. By providing this flexibility, Vex maintains codebase clarity and minimizes overhead, offering a streamlined alternative to languages traditionally designed for large-scale, monolithic systems.

Documentation Table of Contents

Code Setup Environment Profiles	3
Basic Output Scripting Print Statements	4
Variables Global Variables, Local Variables, Variable Operations	5
Conditionals If/Else Statements, Until Statements, While Statements, Repeat Loops	6-7
Functions Creating Functions, Calling Functions	8
User Input Text Input, Button Presses	9
External Libraries Importing Libraries, Using Libraries	10
Other Features and Notes Comments, Encased Value Expansion, Try/Catch Blocks, Loop Control, Script Pausing, Intentional Errors, Ending the Program, Reserved Keywords, Working Around JSON in Variables, Getting Outputs from Libraries	11-13

Documentation written by Gatlin Nicholson
Revision 3
July 8th, 2026

Code Setup

Environment Profiles

The Vex environment defines the operational parameters for code execution. Specifying an environment allows for configurable interpreter sizing, variable system resource allocation, and the restriction of specific language statements.

The 'main' environment is used by default. To explicitly define a target environment at the start of a program, use:

```
env main
```

Environments are system-wide and can be initialized via the terminal. Follow the interactive prompts to configure the environment name, maximum RAM and CPU usage, and restricted statements:

```
$ vex env create
```

```
Ready to create an environment. Please enter the environment's  
variables below:
```

```
NAME > <enter name here>
```

```
MAX-MEM-USAGE (MB) > <enter amount here>
```

```
MAX-CPU-USAGE (%) > <enter amount here>
```

```
BLOCKED-STATEMENTS (comma-separated) > <enter statements here>
```

```
Environment creation is complete. You can now use "<name>"  
across Vex programs on your system.
```

Basic Output Scripting

Print Statements

Print statements are fundamental operations for outputting data to the command line. The following syntax demonstrates a basic string output:

```
print("hello world!")
```

Print statements support the joining of multiple data types, including literal strings and variables. The example below illustrates combining a string literal with the 'username' variable:

```
print("hello, ", username)
```

Expressions, such as arithmetic operations, can be evaluated directly within a print statement without prior variable assignment:

```
print("the answer is ", 1 + 3)
```

Vex provides special escape characters to control the layout of your output within print statements. To start a new line, use the newline character `\n`:

```
print("Line 1\nLine 2")
```

You can also control your output's layout using the tab character. To insert a horizontal tab, use the tab character `\t`.

```
print("Column 1\nColumn 2")
```

Variables

Global Variables

Global variables are accessible throughout the entire script. Variables may store numeric, boolean, or string data types and are defined as follows:

```
globalvar count = 3           # sets "count" to 3
globalvar username = "John"   # sets "username" to "John"
globalvar isGood = True       # sets "isGood" to True
```

Local Variables

Local variables are scoped strictly to the block where they are defined, such as within functions, loops, or conditional statements. The example below initializes 'i' to 0:

```
localvar i = 0
```

Variable Operations

Variables can be modified by applying arithmetic operations directly. By substituting the assignment operator with an arithmetic operator (+, -, *, /, or %), you can update the variable's value. The modulo operator (%) is also supported. The example below demonstrates multiplying "count" by 5:

```
globalvar count * 5
```

Boolean variables can also be modified dynamically by flipping their current state. Using the operator "\$" in place of the assignment operator. This will flip a variable that is True to False, and a variable that is False to True. Nothing is needed after the "\$" operator. In the example below, the boolean variable is flipped like so:

```
globvar isGood $
```

Conditionals

If/Else Statements

Conditional statements facilitate branch execution based on the evaluation of a specified boolean expression. These expressions may involve direct variable comparisons or relational operations. The scope of the conditional block is defined by a single level of indentation following the declaration. Operands (and, or, not, xor, nor, nand) can also be used. The following examples demonstrate standard conditional syntax:

```
if (username = "John")           # executes if username is "John"
    globalvar isJohnHere = True
    print("Welcome, John!")

if (isJohnHere or username = "John")
    globalvar unlockHouse = True
    print("Your house is now unlocked!")
else if (not isJohnHere)         # Executes if isJohnHere is False
    globalvar unlockHouse = False
    print("You aren't John!")
```

Until Statements

Until statements provide a loop mechanism that continues execution as long as the specified condition remains false, terminating immediately upon a true evaluation. These blocks support the use of standard comparison operators (<, >, <=, >=) for numerical data. If a variable is utilized without explicit initialization, it defaults to a value of 0. Below are implementations of terminal and infinite iterative loops:

```
until (i = 5)                    # repeats until "i" equals 5
    localvar i + 1
    print(i)

until (1 = 0)                    # repeats forever, as 1 won't equal 0
    localvar i + 1
    print(i)
```

While Statements

'While' statements function as the inverse of 'until' statements, sustaining the loop execution specifically while the condition maintains a true state. This allows for continuous processing during the validity of a predicate. The following is an example of this behavior:

```
while (i < 10)           # repeats until "i" is greater than 10
  localvar i + 1
  print(i)
```

Repeat Loops

Repeat loops are loops that repeat a segment of code unconditionally and a certain amount of times. Below is an example of a repeat loop:

```
repeat (10)             # repeats 10 times
  print("hello world!")
  localvar timesPrinted + 1
```

Functions

Creating Functions

Functions enable code reuse by encapsulating logical segments, reducing script redundancy, and optimizing system resource usage. They support local variable scoping, which can be dynamically initialized when called. The following example demonstrates a function definition using 'printGreeting' as the identifier.

```
function printGreeting
  repeat (repeatTimes)
    print("hello, ", username, "!")
```

Calling Functions

Once defined, functions can be called throughout a script. You may pass values for any uninitialized local variables during the call, allowing for flexible execution. The following example executes 'printGreeting' by assigning values to the 'repeatTimes' and 'username' variables.

```
dofunc printGreeting(repeatTimes = 5, username = "John")
```


User Input

Text Input

User input allows for the dynamic assignment of variable values at runtime. The following syntax captures string input from the user:

```
globalvar username = input("What is your username: ")
```

Input can be restricted to numeric types to ensure data integrity and prevent type-related runtime errors. This method ignores non-numeric character inputs during entry:

```
globalvar PIN = input.number("What is your PIN: ")
```

Button Presses

Vex utilizes event listeners to monitor hardware or interface interactions. These events are represented as boolean states within specific library namespaces, such as 'event:button-down'. The following implementation monitors for a specific key press:

```
until (1 = 0)  
  if (event:button-down.space)
```

External Libraries

Importing Libraries

Vex is designed as a modular language, with external libraries providing extended functionality for graphical interfaces, advanced mathematics, and system utilities. Libraries are integrated using the 'addlib' statement. The following example imports the 'windowd' library using the 'win' alias:

```
addlib windowd as win
```

Using Libraries

Once a library is imported, its methods and properties are accessible via the defined alias. This example demonstrates initializing a new window using the 'win' alias:

```
win.new(width = 800, height = 500, border = True)
```

Other Features and Notes

Comments

Comments provide inline documentation and facilitate debugging by preventing the execution of designated code segments. Syntax for single and multi-line comments is as follows:

```
# Everything after the comment symbol is now a symbol

# You can also use
# multiple comment symbols
# to make comment blocks!
```

Encased Value Expansion

For improved legibility when passing multiple arguments, Vex supports multi-line expansion for values encased in parentheses:

```
win.new(
    width = 800,
    height = 500,
    border = True
)
```

Try/Catch Blocks

Try/Catch blocks provide exception handling to maintain script execution during runtime errors. The following example demonstrates an iterative loop that persists until valid input is received:

```
until (1 = 0)
    try
        globalvar input = input("What is your username:")
        print(input)
        exitloop
    catch
        print("Invalid input! Try again.")
```

Loop Control

Loop execution can be terminated programmatically using the 'exitloop' statement. This is primarily utilized to break out of infinite iterations when specific logic conditions are met.

Script Pausing

The 'wait' statement suspends script execution for a specified duration in seconds. The following example implements a three-second delay:

```
wait 3
```

Intentional Errors

Programs can also execute their own errors to the command line. These can be warnings, which will not stop the script, or they can be critical, which will stop the script. They help if you want your script to have more verbose errors.

```
warn("Non-emergent error detected.")  
  
critical("Emergent error detected. Stopping execution...")
```

Ending the Program

Programs can be intentionally ended using the 'exit' command.

Reserved Keywords

To prevent errors, try not to use Vex's reserved keywords as variable names or identifiers. Here is a list of the reserved keywords (17): globalvar, localvar, function, dofunc, if, else, until, while, repeat, addlib, try, catch, exitloop, wait, warn, critical, exit.

Working Around JSON in Variables

Using JSON in variables is tricky, especially when you get trailing character errors. This is fixed using ' ' characters at each end of a text string instead of " ". Otherwise, you can still use " ".

Getting Outputs from Libraries

If you wish for your library to output text directly by using one of its statements, the best way to do so is creating a 'tmpOut' (or similar) variable for temporary storage of a library's output. Using the code below, you can directly print a library's output to the terminal. It uses the 'time' library as an example:

```
globalvar tmpOut = time.hms()
```